

Inhalt dieses Projekts

<i>1. Projektübersicht & Geschäftslogik (Info)</i>	<i>2</i>
<i>2. Datenbankstruktur & Vorbereitung (Tabelle erstellen).....</i>	<i>2</i>
<i>3. Datenmodellierung & komplexe Analytik (View erstellen)</i>	<i>3</i>
<i>4. Reporting & Ergebnisanalyse (Daten aus View abfragen)</i>	<i>4</i>
<i>5. Ergebnisanalyse & Visualisierung (Data Output)</i>	<i>4</i>

1. Projektübersicht & Geschäftslogik (Info)

In diesem Projekt präsentiere ich die Aufbereitung einer komplexen Mitarbeiter-Datenbank für analytische Zwecke. Das Ziel ist es, dem Management datengestützte Einblicke in die Gehaltsstruktur und die Karriereentwicklung zu geben. Hierbei werden Rohdaten in wertvolle Business-Insights transformiert.

2. Datenbankstruktur & Vorbereitung (Tabelle erstellen)

Zunächst wurde die Datenstruktur definiert, um eine konsistente Basis für die Analyse zu schaffen. Durch die präzise Definition von Datentypen und Primärschlüsseln wird die Datenintegrität sichergestellt – eine essenzielle Voraussetzung für jede fehlerfreie BI-Auswertung.

```
WITH Anzahl_Befoerderung AS (  
    SELECT  
        emp_no,  
        COUNT(title) AS Anzahl_Beförderungen  
    FROM titles  
    GROUP BY emp_no  
)  
SELECT  
    de.emp_no AS 'Personalnummer',  
    e.first_name AS 'Vorname',  
    e.last_name AS 'Nachname',  
    t.title AS 'Titel',  
    s.salary AS 'Lohn',  
    RANK() OVER(PARTITION BY d.dept_name ORDER BY s.salary DESC) AS  
'Lohn_Rank_Abt',  
    d.dept_name AS 'Abteilung',  
    ROUND(AVG(s.salary) OVER(PARTITION BY d.dept_name)) AS 'Abteilung  
Durchschnitt',  
    ROUND(s.salary - AVG(s.salary) OVER()) AS 'Abweichung von  
Durchschnitt',  
    ab.Anzahl_Beförderungen AS 'Anzahl Beförderungen'  
FROM salaries s  
INNER JOIN dept_emp de ON s.emp_no = de.emp_no  
INNER JOIN departments d ON de.dept_no = d.dept_no  
INNER JOIN titles t ON t.emp_no = s.emp_no  
INNER JOIN employees e ON e.emp_no = s.emp_no  
INNER JOIN Anzahl_Befoerderung ab ON s.emp_no = ab.emp_no  
WHERE de.to_date = '9999-01-01'  
    AND s.to_date = '9999-01-01'  
    AND t.to_date = '9999-01-01'  
ORDER BY d.dept_name ASC, s.salary DESC;
```

3. Datenmodellierung & komplexe Analytik (View erstellen)

*Der Kern dieses Projekts ist die Erstellung einer **VIEW** (siehe **Abbildung 2**). Hierbei habe ich fortgeschrittene SQL-Techniken kombiniert: **Common Table Expressions (CTE)** zur Aggregation der Karrierehistorie sowie **Window Functions (OVER, PARTITION BY)** für den direkten Vergleich von Individualgehältern mit dem Abteilungsdurchschnitt. Durch die Speicherung dieser Logik als **VIEW** wird der Zugriff für Reporting-Tools wie Power BI drastisch vereinfacht.*

```
CREATE VIEW Lohn_database AS
WITH Anzahl_Befoerderung AS (
    SELECT
        emp_no,
        COUNT(title) AS Anzahl_Beförderungen
    FROM titles
    GROUP BY emp_no
)
SELECT
    de.emp_no AS 'Personalnummer',
    e.first_name AS 'Vorname',
    e.last_name AS 'Nachname',
    t.title AS 'Titel',
    s.salary AS 'Lohn',
    RANK() OVER(PARTITION BY d.dept_name ORDER BY s.salary DESC) AS
'Lohn_Rank_Abt',
    d.dept_name AS 'Abteilung',
    ROUND(AVG(s.salary) OVER(PARTITION BY d.dept_name)) AS 'Abteilung
Durchschnitt',
    ROUND(s.salary - AVG(s.salary) OVER()) AS 'Abweichung von
Durchschnitt',
    ab.Anzahl_Befoerderung AS 'Anzahl Beförderungen'
FROM salaries s
INNER JOIN dept_emp de ON s.emp_no = de.emp_no
INNER JOIN departments d ON de.dept_no = d.dept_no
INNER JOIN titles t ON t.emp_no = s.emp_no
INNER JOIN employees e ON e.emp_no = s.emp_no
INNER JOIN Anzahl_Befoerderung ab ON s.emp_no = ab.emp_no
WHERE de.to_date = '9999-01-01'
    AND s.to_date = '9999-01-01'
    AND t.to_date = '9999-01-01';
```

4. Reporting & Ergebnisanalyse (Daten aus View abfragen)

*Schließlich zeige ich, wie effizient die vorbereiteten Daten abgefragt werden können. Mit minimalem Code-Aufwand lassen sich nun komplexe Fragen beantworten, wie zum Beispiel die Identifikation der Top-Verdiener (Top-Performer) pro Abteilung (siehe **Abbildung 3**). Dies bildet die perfekte Grundlage für automatisierte Dashboards.*

```
SELECT
    Personalnummer,
    Vorname,
    Nachname,
    Abteilung,
    Lohn AS 'Top-Lohn'
FROM Lohn_database
WHERE Lohn_Rank_Abt = 1
ORDER BY Lohn DESC;
```

5. Ergebnisanalyse & Visualisierung (Data Output)

Im letzten Schritt präsentiere ich das Ergebnis der Datenabfrage. Der Screenshot zeigt den extrahierten Datensatz aus der vorbereiteten View. Hier wird deutlich, wie die zuvor definierten Window Functions (Ranking und Durchschnittsberechnung) in der Praxis funktionieren. Dieser saubere Datenausgang dient als ideale Grundlage für die Erstellung von Dashboards in Power BI oder für direkte Business-Reports an das Management.

Abbildung 1:
EER-Diagramm der Datenbankstruktur und Relationen.

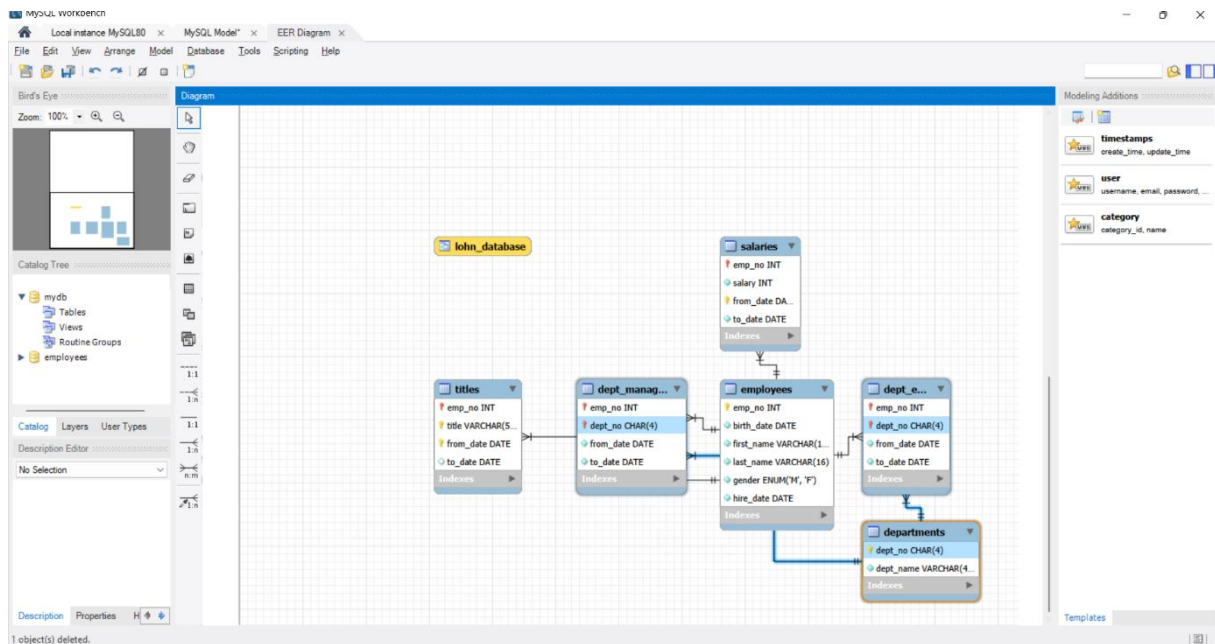


Abbildung 2:
Definition der analytischen View und erfolgreiche Ausführung im SQL-Editor.

```

11  e.first_name AS 'Vorname',
12  e.last_name AS 'Nachname',
13  t.title AS 'titel',
14  s.salary AS 'Lohn',
15  rank() over(partition by d.dept_name order by s.salary desc) AS 'Lohn_Rank_Abt',
16  d.dept_name AS 'Abteilung',
17  round(AVG(salary) OVER(partition by d.dept_name)) AS 'Abteilung Durchschnitt',
18  round(s.salary - AVG(s.salary) OVER()) AS 'Abweichung von Durchschnitt',
19  ab.Anzahl_Befoerderung AS 'Anzahl Befoerderungen'
20 FROM salaries s
21 INNER JOIN dept_emp de ON s.emp_no = de.emp_no
22 INNER JOIN departments d ON de.dept_no = d.dept_no
23 INNER JOIN titles t ON t.emp_no = s.emp_no
24 INNER JOIN employees e ON e.emp_no = s.emp_no
25 INNER JOIN Anzahl_Befoerderung ab ON s.emp_no = ab.emp_no
26
27 WHERE de.to_date = '9999-01-01'
28 AND s.to_date = '9999-01-01'
29 AND t.to_date = '9999-01-01'
30 order by d.dept_name asc, s.salary desc;
31

```

The screenshot shows the MySQL Workbench SQL Editor with the definition of an analytical view named 'lohn_analyse'. The SQL code defines a view that calculates salary ranks and averages by department. The output window shows the successful execution of the CREATE VIEW statement.

Abbildung 3:
Extraktion der Top-Verdiener pro Abteilung inklusive Gehalts-Benchmarks

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays a tree view of the database structure, including tables like 'employees', 'departments', and 'titles'. The main editor window contains a SQL query that filters for the top 10 highest earners in each department as of January 1, 1999. The query is as follows:

```
28 AND s.to_date = '9999-01-01'
29 AND t.to_date = '9999-01-01'
30 order by d.dept_name asc, s.salary desc;
31
32 • SELECT
33     Personalnummer,
34     Vorname,
35     Nachname,
36     Abteilung,
37     Lohn AS 'TopLohn'
38 FROM
39     Lohn_database
40 WHERE
41     Lohn_Rank_Abt = 1
42 ORDER BY Lohn DESC;
43
44
```

Below the query editor, the 'Result Grid' displays the results of the query. The table has five columns: 'Personalnummer', 'Vorname', 'Nachname', 'Abteilung', and 'TopLohn'. The results are sorted by salary in descending order within each department.

Personalnummer	Vorname	Nachname	Abteilung	TopLohn
43624	Tokuyasu	Pesch	Sales	158220
18006	Vidya	Hanabata	Customer Service	144866
13386	Khosrow	Sgarro	Development	144434
89825	Ingmar	Isard	Marketing	143644
28124	Adhemar	Eiter	Finance	134662
107632	Fumino	Schahn	Production	123552
34947	Mario	Perflyeva	Research	124181
48595	Shan	Encarnacion	Human Resources	123674
73644	Eberhardt	Badhouse	Quality Management	121899